

Строка параметров и Карта параметров

[Прыжок на дно](#)

Питер Диллинджер отредактировал эту страницу 30 марта 2022 г. · [14 правок](#)

Пользователи передают параметры в RocksDB через класс *Options*. Помимо установки параметров в классе *Options*, есть еще два способа:

1. получить класс параметров из [файла параметров](#).
2. получить его из строки параметров
3. получить его из строковой карты

↪ Строка параметров

Чтобы получить параметр из строки, вызовите вспомогательную функцию `GetColumnFamilyOptionsFromString()` или `GetDBOptionsFromString()` со строкой, содержащей информацию. Также существуют специальные функции `GetBlockBasedTableOptionsFromString()` и `GetPlainTableOptionsFromString()` для получения параметров конкретной таблицы.

Пример строки с опциями:

```
table_factory=PlainTable;prefix_extractor=rocksdb.CappedPrefix.13;compara
```

Каждый вариант будет представлен в виде `<option_name>:<option_value>`, разделенных `;`. Чтобы ознакомиться со списком поддерживаемых вариантов, перейдите в [раздел ниже](#).

↪ Карта параметров

Аналогичным образом пользователи могут получить классы опций из строковой карты, вызвав вспомогательную функцию `GetColumnFamilyOptionsFromMap()`, `GetDBOptionsFromMap()`, `GetBlockBasedTableOptionsFromMap()` или `GetPlainTableOptionsFromMap()`. Передается строковая карта, которая сопоставляет имя опции и ее значение в виде обычной текстовой строки. Пример строковой карты для опции:



```
std::unordered_map<std::string, std::string> cf_options_map = {
{"write_buffer_size", "1"},
{"max_write_buffer_number", "2"},
{"min_write_buffer_number_to_merge", "3"},
{"max_write_buffer_number_to_maintain", "99"},
{"сжатие", "kSnappyCompression"},
{"compression_per_level",
"kNoCompression:"
    "kSnappyCompression:"
    "kZlibCompression:"
    "kBZip2Compression:"
    "kLZ4Compression:"
    "kLZ4HCCompression:"
    "kXpressCompression:"
    "kZSTD:"
    "kZSTDNotFinalCompression"},
{"bottommost_compression", "kLZ4Compression"},
{"compression_opts", "4:5:6:7"},
{"num_levels", "8"},
{"level0_file_num_compaction_trigger", "8"},
{"level0_slowdown_writes_trigger", "9"},
{"level0_stop_writes_trigger", "10"},
{"target_file_size_base", "12"},
{"target_file_size_multiplier", "13"},
{"max_bytes_for_level_base", "14"},
{"level_compaction_dynamic_level_bytes", "правда"},
{"max_bytes_for_level_multiplier", "15.0"},
{"max_bytes_for_level_multiplier_additional", "16:17:18"},
{"max_compaction_bytes", "21"},
{"soft_rate_limit", "1.1"},
{"hard_rate_limit", "2.1"},
{"hard_pending_compaction_bytes_limit", ""211"},
{"arena_block_size", "22"},
{"отключаем_авто_компакции", "true"},
{"compaction_style", "kcompactionstylelevel"},
{"verify_checksums_in_compaction", "false"},
{"compaction_options_fifo", "23"},
{"max_sequential_skip_in_iterations", "24"},
{"inplace_update_support", "true"},
{"report_bg_io_stats", "правда"},
{"compaction_measure_io_stats", "ложное"},
{"inplace_update_num_locks", "25"},
{"memtable_prefix_bloom_size_ratio", "0.26"},
{"memtable_huge_page_size", "28"},
{"bloom_locality", "29"},
{"max_successive_merges", "30"},
{"min_partial_merge_operands", "31"},
{"prefix_extractor", "исправлено:31"},
{"optimize_filters_for_hits", "правда"},
};
```

To find the list of options supported, check [the section below](#).

🔗 Find the supported options in option string and option map

In both of option string and option map, option name maps the variable names in the target class, `DBOptions`, `ColumnFamilyOptions`, `BlockBasedTableOptions`, or `PlainTableOptions`. For `DBOptions` and `ColumnFamilyOptions`, you can find the list of them and their descriptions in two respective classes in the source file [options.h](#) of the source code of your release. For the other two options, you can find them in file [table.h](#)

Note, although most of the options in the option class are supported in the option string, there are exceptions. You can find the list of supported options in variable `db_options_type_info`, `cf_options_type_info` and `block_based_table_type_info` in the source file [options/options_helper.h](#) of the source code of your release.

If the option is a callback class, e.g. comparators, compaction filter, and merge operators, you will usually need to pass the pointer of the callback class as the value, which will be casted in to an object.

There are exception. Some special callback classes are supported by the option string or map:

- Prefix extractor (option name `prefix_extractor`), whose value can be passed as `rocksdb.FixedPrefix.<prefix_length>` or `rocksdb.CappedPrefix.<prefix_length>`.
- Filter policy (option name `filter_policy`), whose value can be passed as `bloomfilter:<bits_per_key>`
- Table factory (option name `table_factory`). The values will be either `BlockBasedTable` or `PlainTable`. Other than that, two special option string names are used to provide the options, `block_based_table_factory` or `plain_table_factory`. The value of the options will be the option string of `BlockBasedTableOptions` or `PlainTableOptions`.
- Memtable Factory (option name `memtable_factory`). It can take value of `skip_list`, `prefix_hash`, `hash_linkedlist`, `vector` or `cuckoo`.

🔗 SetOptions

The functions `DB::SetOptions` and `DB::SetDBOptions` support changing some configuration parameters dynamically using an options map of settings you would like to change.

Contents

- [RocksDB Wiki](#)
- [Overview](#)
- [RocksDB FAQ](#)
- [Terminology](#)
- [Requirements](#)
- [Contributors' Guide](#)
- [Release Methodology](#)
- [RocksDB Users and Use Cases](#)
- [RocksDB Public Communication and Information Channels](#)
- [Basic Operations](#)
 - [Iterator](#)
 - [Prefix seek](#)
 - [SeekForPrev](#)
 - [Tailing Iterator](#)
 - [Compaction Filter](#)
 - [Multi Column Family Iterator](#)
 - [Read-Modify-Write \(Merge\) Operator](#)
 - [Column Families](#)
 - [Creating and Ingesting SST files](#)
 - [Single Delete](#)
 - [SST Partitioner](#)
 - [Low Priority Write](#)
 - [Time to Live \(TTL\) Support](#)
 - [Transactions](#)
 - [Snapshot](#)
 - [DeleteRange](#)
 - [Atomic flush](#)
 - [Read-only and Secondary instances](#)
 - [Approximate Size](#)
 - [User-defined Timestamp](#)
 - [Wide Columns](#)
 - [BlobDB](#)
 - [Online Verification](#)
- **Options**
 - [Setup Options and Basic Tuning](#)
 - [Option String and Option Map](#)
 - [RocksDB Options File](#)
- [MemTable](#)
- [Journal](#)
 - [Write Ahead Log \(WAL\)](#)
 - [Write Ahead Log File Format](#)
 - [WAL Recovery Modes](#)
 - [WAL Performance](#)
 - [WAL Compression](#)
 - [MANIFEST](#)
 - [Track WAL in MANIFEST](#)
- **Cache**
 - [Block Cache](#)

- [SecondaryCache \(Experimental\)](#)
- [Write Buffer Manager](#)
- [Compaction](#)
 - [Leveled Compaction](#)
 - [Universal compaction style](#)
 - [FIFO compaction style](#)
 - [Manual Compaction](#)
 - [Subcompaction](#)
 - [Choose Level Compaction Files](#)
 - [Managing Disk Space Utilization](#)
 - [Trivial Move Compaction](#)
 - [Remote Compaction](#)
- **SST File Formats**
 - [Block-based Table Format](#)
 - [PlainTable Format](#)
 - [CuckooTable Format](#)
 - [External Table](#)
 - [Index Block Format](#)
 - [Bloom Filter](#)
 - [Data Block Hash Index](#)
- [IO](#)
 - [Rate Limiter](#)
 - [SST File Manager](#)
 - [Direct I/O](#)
- [Compression](#)
 - [Dictionary Compression](#)
- [Full File Checksum and Checksum Handoff](#)
- [Background Error Handling](#)
- [Huge Page TLB Support](#)
- [Tiered Storage \(Experimental\)](#)
- **Logging and Monitoring**
 - [Logger](#)
 - [Statistics](#)
 - [Compaction Stats and DB Status](#)
 - [Perf Context and IO Stats Context](#)
 - [EventListener](#)
- [Known Issues](#)
- [Troubleshooting Guide](#)
- [Tests](#)
 - [Stress Test](#)
 - [Fuzzing](#)
 - [Benchmarking](#)
- **Tools / Utilities**
 - [Administration and Data Access Tool](#)
 - [How to Backup RocksDB?](#)
 - [Replication Helpers](#)
 - [Checkpoints](#)
 - [How to persist in-memory RocksDB database](#)
 - [Third-party language bindings](#)
 - [RocksDB Trace, Replay, Analyzer, and Workload Generation](#)
 - [Block cache analysis and simulation tools](#)

- [IO Tracer and Parser](#)
- **Implementation Details**
 - [Delete Stale Files](#)
 - [Partitioned Index/Filters](#)
 - [WritePrepared-Transactions](#)
 - [WriteUnprepared-Transactions](#)
 - [How we keep track of live SST files](#)
 - [How we index SST](#)
 - [Merge Operator Implementation](#)
 - [RocksDB Repairer](#)
 - [Write Batch With Index](#)
 - [Two Phase Commit](#)
 - [Iterator's Implementation](#)
 - [Simulation Cache](#)
 - [\[To Be Deprecated\] Persistent Read Cache](#)
 - [DeleteRange Implementation](#)
 - [unordered_write](#)
- **Extending RocksDB**
 - [RocksDB Configurable Objects](#)
 - [The Customizable Class](#)
 - [Object Registry](#)
- **RocksJava**
 - [RocksJava Basics](#)
 - [Logging in RocksJava](#)
 - [JNI Debugging](#)
 - [RocksJava API TODO](#)
 - [RocksJava Performance on Flash Storage](#)
 - [Tuning RocksDB from Java](#)
- **Performance**
 - [Performance Benchmarks](#)
 - [In Memory Workload Performance](#)
 - [Read-Modify-Write \(Merge\) Performance](#)
 - [Delete A Range Of Keys](#)
 - [Write Stalls](#)
 - [Pipelined Write](#)
 - [MultiGet Performance](#)
 - [Tuning Guide](#)
 - [Memory usage in RocksDB](#)
 - [Speed-Up DB Open](#)
 - [Implement Queue Service Using RocksDB](#)
 - [Asynchronous IO](#)
 - [Off-peak in RocksDB](#)
- [Projects Being Developed](#)
- **Misc**
 - [Building on Windows](#)
 - [Developing with an IDE](#)
 - [Open Projects](#)
 - [Talks](#)
 - [Publication](#)
 - [Features Not in LevelDB](#)

- [How to ask a performance-related question?](#)
- [Articles about Rocks](#)

Clone this wiki locally

`https://github.com/facebook/rocksdb.wiki.git`

